



ENSIM
École d'ingénieurs
Le Mans Université



Compte rendu - Application et micro-projet : gestion de température sur PIC16F1719

4A ASTRE - MICROCONTROLEUR PIC
ENSIM - LE MANS UNIVERSITE
PROGRAMMATION MICROCONTROLEUR EN LANGAGE C

ESSOMBA MICHAEL

Année universitaire : 2025–2026

Tuteur(s) université :
EMMANUEL MALANDIN



**Le Mans
Université**

Table des matières

Resume	3
1 Contexte du TP	3
2 Matériel utilise	3
2.1 Carte Explorer 8	3
2.2 Microcontrôleur PIC16F1719	4
2.3 Capteur TMP36	5
2.4 Composants externes	6
3 Choix des broches et justification	7
4 Principe de conversion analogique numérique	8
4.1 Resolution ADC	8
4.2 Moyennage des mesures	8
5 Application de base : affichage de la temperature	9
5.1 Objectif	9
5.2 Algorithme	9
5.3 Organigramme	10
5.4 Code C de l'application de base	10
6 Micro-projet sans PWM	11
6.1 Objectif	11
6.2 Regles de fonctionnement	11
6.3 Algorithme	11
6.4 Organigramme du micro-projet sans PWM	13
6.5 Code C verifie : micro-projet simple	13
7 Extension PWM	15
7.1 Correction de la logique PWM	15
7.2 Principe de la PWM	15
7.3 Routage PPS vers RB1	15
7.4 Code C final avec logique PWM corrigeée	16
8 Configurations utilisees	18
8.1 Fichiers MCC et bibliotheques	18
8.2 Registres importants	18
9 Schemas et cablage	19
10 Analyse des resultats experimentaux	22
10.1 Affichage LCD	22
10.2 Cohérence avec la position du potentiometre	24

11 Points de verification et corrections realisees	24
11.1 Probleme de variation de temperature avec la consigne	24
11.2 Correction de la logique des LEDs	25
12 Conclusion	25
References documentaires	25

Resume

Ce compte rendu presente la realisation d'une application puis d'un micro-projet de regulation de temperature sur carte Explorer 8 equipee d'un microcontrôleur PIC16F1719. Le travail consiste a mesurer une temperature a l'aide d'un capteur TMP36, a lire une consigne fournie par le potentiometre de la carte, puis a afficher les informations sur un ecran LCD et piloter des LEDs selon l'etat du systeme. Une extension ajoute une commande PWM sur la LED rouge lorsque la temperature depasse de plus de 10% la consigne.

Le document contient le contexte technique, le choix des broches, les algorithmes, les organigrammes, les codes C verifiés, les captures de tests et une analyse des resultats obtenus.

1 Contexte du TP

Le sujet demande de developper une application en langage C permettant d'afficher la temperature ambiante en degres Celsius sur un LCD. La consigne est donnee par un potentiometre et l'etat du systeme est indique par des LEDs. Le mini-projet etend cette application en comparant la temperature actuelle a une consigne reglable, puis en activant une alarme rouge au-dela d'un depassement de 10%.

Les objectifs sont les suivants :

- lire une tension analogique issue d'un capteur TMP36 ;
- convertir cette tension en temperature ;
- lire le potentiometre de consigne ;
- afficher temperature et consigne sur le LCD ;
- commander trois LEDs : verte, jaune, rouge ;
- ajouter une modulation PWM sur la LED rouge lorsque le seuil critique est atteint.

2 Matériel utilise

2.1 Carte Explorer 8

La carte Explorer 8 est une carte de developpement Microchip prevue pour des microcontrôleurs PIC 8 bits. Elle integre des supports de microcontrôleurs, des connecteurs d'extension, un LCD, un potentiometre, des boutons et des connecteurs de programmation. La documentation de la carte indique egalement la presence d'elements materiels comme l'afficheur LCD, les interrupteurs, l'entree analogique par potentiometre et les connecteurs d'extension.

Explorer 8 Development Board User's Guide

1.3 EXPLORER 8 DEVELOPMENT BOARD

The Explorer 8 Development Board has the following hardware features. Each feature's number corresponds to the number in Figure 1-1, showing the feature's location on the board:

1. MCP2221 USB-to-UART/I²C serial converter (U3)
2. 16x2 Character LCD (LCD1)
3. MCP23S17 I/O Expander for LCD Interface (U4)
4. Eight blue LEDs (D1 to D8).
5. Male header pins for attaching Plug-in-Modules (U1A)
6. Socket for attaching 40-pin PIC MCUs (J8)
7. Socket for attaching 28-pin PIC MCUs (J13)
8. Socket for attaching 8/14/20-pin PIC MCUs (J10)
9. mikroBUS headers for attaching mikroBUS compatible boards (J32 and J35)
10. Two push button switches for external stimulus (S1 and S2)
11. PICtail Expansion Connectors for PICtail Daughter Boards (J3, J5, J11 and J28)
12. PICtail Plus Card Edge Modular Expansion Connectors for PICtail Plus Daughter Boards (J19)
13. Test Points for 5.0V (TP1 and TP7), 3.3V (TP6) and V_{VAR} (TP5). V_{VAR} is the variable voltage and is equal to the device V_{DD} and its associated logic
14. 20-pin in-line expansion connector (J33)
15. 8 MHz crystal for device external oscillator (Y1)
16. Digilent Pmod compatible connectors (J17 and J20)
17. 10 kΩ Potentiometer for analog inputs (R25)
18. Variable Power Indicator LED (D9)
19. 3.3V Power Indicator (LD2)
20. 5.0V Power Indicator (LD1)
21. 5.0V (U5), 3.3V (U1) and Variable (U2) power supplies
22. Barrel connector for 9V DC Supply (J1)
23. RJ11 connector for ICSP programming/debugging using REAL ICE and ICD 3 (J26)
24. 6-pin male header for ICSP programming/debugging using PICKIT 3 (J12)
25. Push button switch on MCLR for external Reset (S3)
26. Micro USB socket for USB communication and/or USB power (J18)

FIGURE 2 – Vue generale de la carte Explorer 8, issue de la documentation Microchip.

2.2 Microcontrôleur PIC16F1719

Le PIC16F1719 appartient à la famille PIC16(L)F1717/8/9. Il dispose notamment d'un cœur RISC 8 bits, d'un ADC 10 bits, de timers, de PWM, d'un module MSSP, d'un EUSART et du PPS permettant de router certaines fonctions périphériques vers des broches configurables. Cette souplesse est importante pour associer le PWM à la broche RB1 dans le projet.

PIC16(L)F1717/8/9

FIGURE 1-2: PIC16(L)F1717/9 BLOCK DIAGRAM

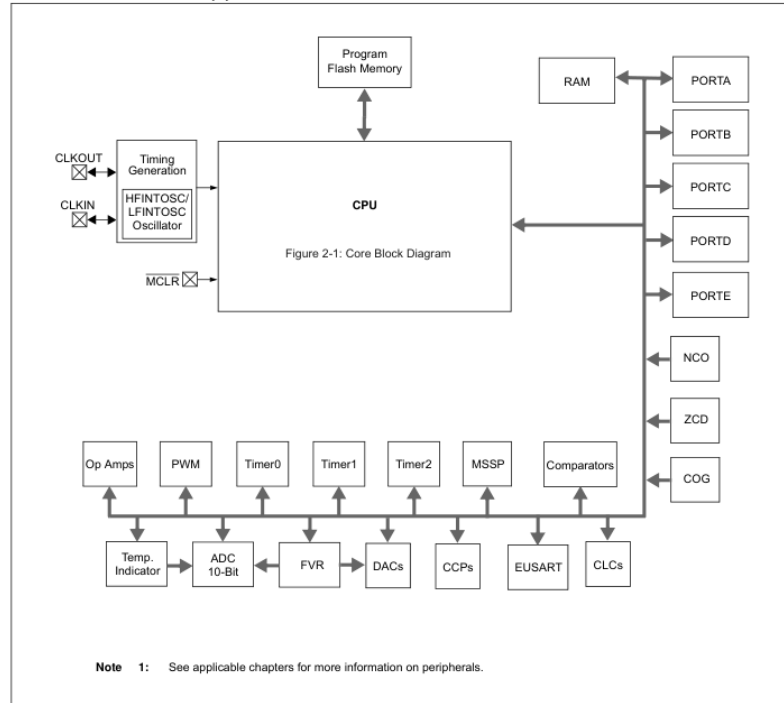


FIGURE 3 – Architecture interne generale du PIC16F1717/9 : CPU, memoire, ports et peripheriques.

2.3 Capteur TMP36

Le TMP36 est un capteur de temperature analogique. Sa sortie est lineaire avec une pente de 10 mV/°C. Pour le TMP36, la tension de sortie vaut environ 750 mV a 25 °C. La relation utilisee dans le programme est :

$$T(^{\circ}C) = \frac{V_{out}(mV) - 500}{10}$$

avec un offset de 500 mV correspondant à 0°C.



Low Voltage Temperature Sensors

TMP35/TMP36/TMP37

FEATURES

Low Voltage Operation (2.7 V to 5.5 V)
Calibrated Directly in °C
10 mV/°C Scale Factor (20 mV/°C on TMP37)
±2°C Accuracy over Temperature (Typ)
±0.5°C Linearity (Typ)
Stable with Large Capacitive Loads
Specified -40°C to +125°C, Operation to +150°C
Less than 50 µA Quiescent Current
Shutdown Current 0.5 µA Max
Low Self-Heating

APPLICATIONS

Environmental Control Systems
Thermal Protection
Industrial Process Control
Fire Alarms
Power System Monitors
CPU Thermal Management

PRODUCT DESCRIPTION

The TMP35, TMP36, and TMP37 are low voltage, precision centigrade temperature sensors. They provide a voltage output that is linearly proportional to the Celsius (Centigrade) temperature. The TMP35/TMP36/TMP37 do not require any external calibration to provide typical accuracies of ±1°C at +25°C and ±2°C over the -40°C to +125°C temperature range.

The low output impedance of the TMP35/TMP36/TMP37 and its linear output and precise calibration simplify interfacing to temperature control circuitry and A/D converters. All three devices are intended for single-supply operation from 2.7 V to 5.5 V maximum. Supply current runs well below 50 µA, providing very low self-heating—less than 0.1°C in still air. In addition, a shutdown function is provided to cut supply current to less than 0.5 µA.

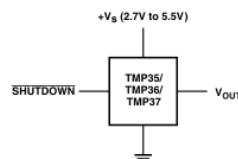
The TMP35 is functionally compatible with the LM35/LM45 and provides a 250 mV output at 25°C. The TMP35 reads temperatures from 10°C to 125°C. The TMP36 is specified from -40°C to +125°C, provides a 750 mV output at 25°C, and operates to +125°C from a single 2.7 V supply. The TMP36 is functionally compatible with the LM50. Both the TMP35 and TMP36 have an output scale factor of 10 mV/°C. The TMP37 is intended for applications over the range 5°C to 100°C and provides an output scale factor of 20 mV/°C. The TMP37 provides a 500 mV output at 25°C. Operation extends to 150°C with reduced accuracy for all devices when operating from a 5 V supply.

The TMP35/TMP36/TMP37 are all available in low cost 3-lead TO-92, SOIC-8, and 5-lead SOT-23 surface-mount packages.

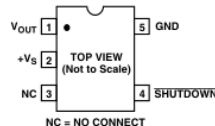
REV. C

Information furnished by Analog Devices is believed to be accurate and reliable. However, no responsibility is assumed by Analog Devices for its use, nor for any infringements of patents or other rights of third parties that may result from its use. No license is granted by implication or otherwise under any patent or patent rights of Analog Devices.

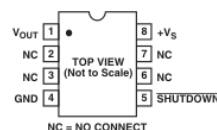
FUNCTIONAL BLOCK DIAGRAM



PACKAGE TYPES AVAILABLE RT-5 (SOT-23)



RN-8 (SOIC)



TO-92



PIN 1, +Vs; PIN 2, Vout; PIN 3, GND

FIGURE 4 – Brochage et principe de fonctionnement du TMP35/TMP36/TMP37.

2.4 Composants externes

Les composants utilisés sur breadboard sont :

— un capteur TMP36;

- trois LEDs : verte, jaune et rouge ;
- trois resistances de limitation de courant ;
- des fils de liaison vers la carte Explorer 8 ;
- le potentiometre R25 deja present sur la carte pour fixer la consigne.

3 Choix des broches et justification

Le choix final des broches tient compte des observations de cablage realisees pendant le TP. La lecture de temperature a d'abord ete testee sur plusieurs entrees analogiques. La broche finalement retenue est **RC2/AN14**, car les mesures du TMP36 etaient stables sur cette entree, contrairement aux essais precedents ou la temperature variait avec le potentiometre.

TABLE 1 – Affectation des broches dans le projet final

Fonction	Broche PIC	Canal/Signal	Justification
TMP36	RC2	AN14	Entree analogique stable, separee du potentiometre
Potentiometre R25	RA0	AN0	Entree analogique de consigne deja disponible sur Explorer 8
LED rouge	RB1	PWM2 via PPS	Sortie associee a la modulation PWM
LED verte	RB2	GPIO	Indique temperature inferieure a la consigne
LED jaune	RB3	GPIO	Indique temperature superieure a la consigne sans alarme
LCD	Connecteur LCD carte	Bus LCD	PWM Affichage temperature, consigne et etat

La table d'allocation du PIC16F1719 confirme que RC2 peut etre utilise en entree analogique AN14 et que RA0 correspond au canal AN0. Elle indique egalement que les ports B peuvent etre utilises comme entrees/sorties numeriques et que les sorties alternatives des peripheriques peuvent etre associees par PPS.

DS400017/40B-page 8

TABLE 2: 40/44-PIN ALLOCATION TABLE (PIC16(L)F1717/9)

I/O ⁽¹⁾	PDP	TQFP	UGFN	ADC	Reference	Comparator	Op Amp	DAC	Zero Cross	Timers	CCP	NCO	PWM	COG	MSSP	EUSART	CLC	Interrupt	Pullup	Basic
RA0	2	19	17	AN0		C1IN0- C2IN0-											CLCIN0 ⁽²⁾	IOC	Y	
RA1	3	20	18	AN1		C1IN1- C2IN1-	OPA1OUT										CLCIN1 ⁽²⁾	IOC	Y	
RA2	4	21	19	AN2	V _{REF} ⁺	C1IN0+ C2IN0+		DAC1OUT1												
RA3	5	22	20	AN3	V _{REF} ⁺	C1IN1+														
RA4	6	23	21				OPA1IN+													
RA5	7	24	22	AN4			OPA1IN-	DAC2OUT1		T0CK ⁽¹⁾						nSS ⁽¹⁾				
RA6	14	31	29																	
RA7	13	30	28																	
RB0	33	8	8	AN12		C2IN1+			ZCD					COG1IN ⁽¹⁾						
RB1	34	9	9	AN10		C1IN3- C2IN3-	OPA2OUT													
RB2	35	10	10	AN8			OPA2IN-													
RB3	36	11	11	AN9		C1IN2- C2IN2-	OPA2IN+													
RB4	37	14	12	AN11																
RB5	38	15	13	AN13						T1G ⁽¹⁾										
RB6	39	16	14																	
RB7	40	17	15					DAC1OUT2 DAC2OUT2												
RC0	15	32	30							T1CK ⁽¹⁾⁽²⁾ SOSC ⁽¹⁾										
RC1	16	35	31							CCP2 ⁽¹⁾										
RC2	17	36	32	AN14						CCP1 ⁽¹⁾										
RC3	18	37	33	AN15												SCL/SCK ⁽¹⁾				

Note 1: Default peripheral input. Alternate pins can be selected as the peripheral input with the PPS input selection registers.
 2: All pin digital outputs default to PORT latch data. Alternate outputs can be selected as the peripheral digital output with the PPS output selection registers.
 3: These peripheral functions are bidirectional. The output pin selections must be the same as the input pin selections.
 4: Alternate outputs are excluded from solid shaded areas.
 5: Alternate inputs are excluded from dot shaded areas.

PIC16(L)F1717/8/9

© 2014-2015 Microchip Technology Inc.

FIGURE 5 – Extrait de la table d'allocation des broches du PIC16F1717/9. RC2 est associe a AN14, RA0 a AN0.

4 Principe de conversion analogique numerique

4.1 Resolution ADC

L'ADC du PIC16F1719 fournit une valeur codee sur 10 bits, donc comprise entre 0 et 1023. Avec une reference de 5V, la tension mesuree est estimee par :

$$V = \frac{ADC \times 5}{1023}$$

Puis, pour le TMP36 :

$$T = (V - 0.5) \times 100$$

4.2 Moyennage des mesures

Un moyennage sur 8 conversions est utilise pour stabiliser les mesures. Cette solution reduit les fluctuations dues au bruit, aux changements de canal ADC et aux effets de charge du condensateur interne d'echantillonnage.

Listing 1 – Fonction de lecture ADC moyennée

```

1 uint16_t Lire_ADC_Moyenne(uint8_t canal)
2 {
3     uint32_t somme = 0;
4
5     ADCON0bits.CHS = canal;
6
7     for(uint8_t i = 0; i < 8; i++)
8     {
9         __delay_us(50);
10        ADCON0bits.GO_nDONE = 1;
11        while(ADCON0bits.GO_nDONE);
12        somme += ((uint16_t)ADRESH << 8) | ADRESL;
13    }
14
15    return (uint16_t)(somme / 8);
16 }

```

5 Application de base : affichage de la temperature

5.1 Objectif

L'application de base consiste à lire le TMP36, convertir la tension en degrés Celsius, puis afficher la temperature sur le LCD. C'est une étape de validation avant l'ajout de la consigne et des LEDs.

5.2 Algorithme

Listing 2 – Algorithme de l'application de base

```

1 Initialiser le systeme
2 Initialiser le LCD
3 Initialiser l'ADC
4 Configurer RC2 en entree analogique AN14
5
6 Tant que vrai :
7     Lire AN14
8     Convertir ADC en tension
9     Convertir tension en temperature
10    Afficher temperature sur le LCD
11    Attendre 500 ms

```

5.3 Organigramme

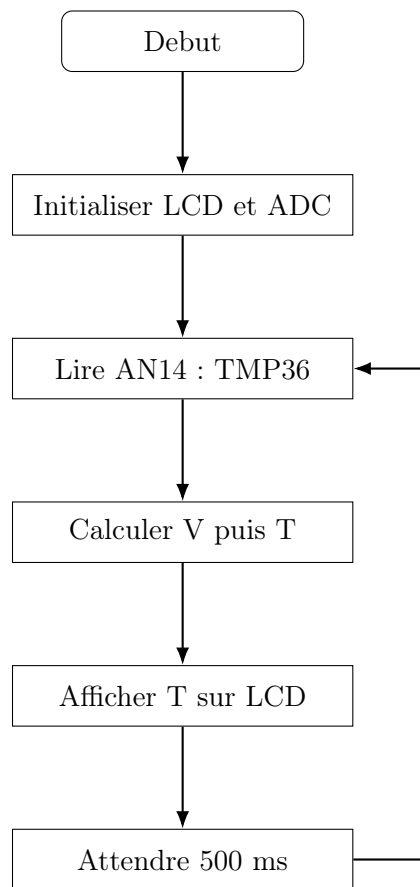


FIGURE 6 – Organigramme de l'application de base.

5.4 Code C de l'application de base

Listing 3 – Application de base : lecture TMP36 et affichage LCD

```

1  #include "mcc.h"
2  #include "lcd.h"
3  #include <stdio.h>
4  #include <stdint.h>
5
6  #define TMP36_CHANNEL 14      // RC2 = AN14
7
8  uint16_t Lire_ADC_Moyenne(uint8_t canal)
9  {
10     uint32_t somme = 0;
11     ADCON0bits.CHS = canal;
12
13     for(uint8_t i = 0; i < 8; i++)
14     {
15         __delay_us(50);
16         ADCON0bits.GO_nDONE = 1;
17         while(ADCON0bits.GO_nDONE);
18         somme += ((uint16_t)ADRESH << 8) | ADRESL;
19     }
20     return (uint16_t)(somme / 8);
  
```

```

21 }
22
23 void main(void)
24 {
25     uint16_t adcTemp;
26     float voltage;
27     float temperature;
28     char buffer[17];
29
30     SYSTEM_Initialize();
31     LCD_Initialize();
32     LCD_Clear();
33
34     ADCON1bits.ADFM = 1;
35     TRISCbits.TRISC2 = 1;
36     ANSELbits.ANSC2 = 1;
37
38     while(1)
39     {
40         adcTemp = Lire_ADC_Moyenne(TMP36_CHANNEL);
41         voltage = ((float)adcTemp * 5.0) / 1023.0;
42         temperature = (voltage - 0.5) * 100.0;
43
44         LCD_Clear();
45         LCD_GoTo(0,0);
46         LCD_WriteString((const uint8_t*)"Temperature");
47         LCD_GoTo(1,0);
48         sprintf(buffer, "T=%.1f C", temperature);
49         LCD_WriteString((const uint8_t*)buffer);
50
51         __delay_ms(500);
52     }
53 }

```

6 Micro-projet sans PWM

6.1 Objectif

Le micro-projet associe la température actuelle et la température de consigne. La température actuelle provient du TMP36 et la consigne est fixée par le potentiomètre R25. La consigne est ramenée dans la plage 15 °C à 45 °C. Cette plage évite les consignes trop basses et correspond mieux à une application de régulation ambiante.

6.2 Regles de fonctionnement

TABLE 2 – Regles de commande sans PWM

Condition	LED active	Message LCD
$T < C$	Verte	LED VERTE
$C \leq T < 1.1C$	Jaune	LED JAUNE
$T \geq 1.1C$	Rouge	LED ROUGE

6.3 Algorithme

Listing 4 – Algorithme du micro-projet sans PWM

```

1 Initialisation :
2   LCD
3   ADC
4   LEDs
5   AN14 pour TMP36
6   AN0 pour potentiometre
7
8 Boucle infinie :
9   Lire AN14
10  Calculer temperature T
11  Lire AN0
12  Calculer consigne C entre 15 et 45 degres
13  Calculer seuil_rouge = 1.10 * C
14
15  Eteindre toutes les LEDs
16  Si T < C :
17      LED verte ON
18  Sinon si T < seuil_rouge :
19      LED jaune ON
20  Sinon :
21      LED rouge ON
22
23  Afficher T, C et l'etat sur LCD
24  Attendre 500 ms

```

6.4 Organigramme du micro-projet sans PWM

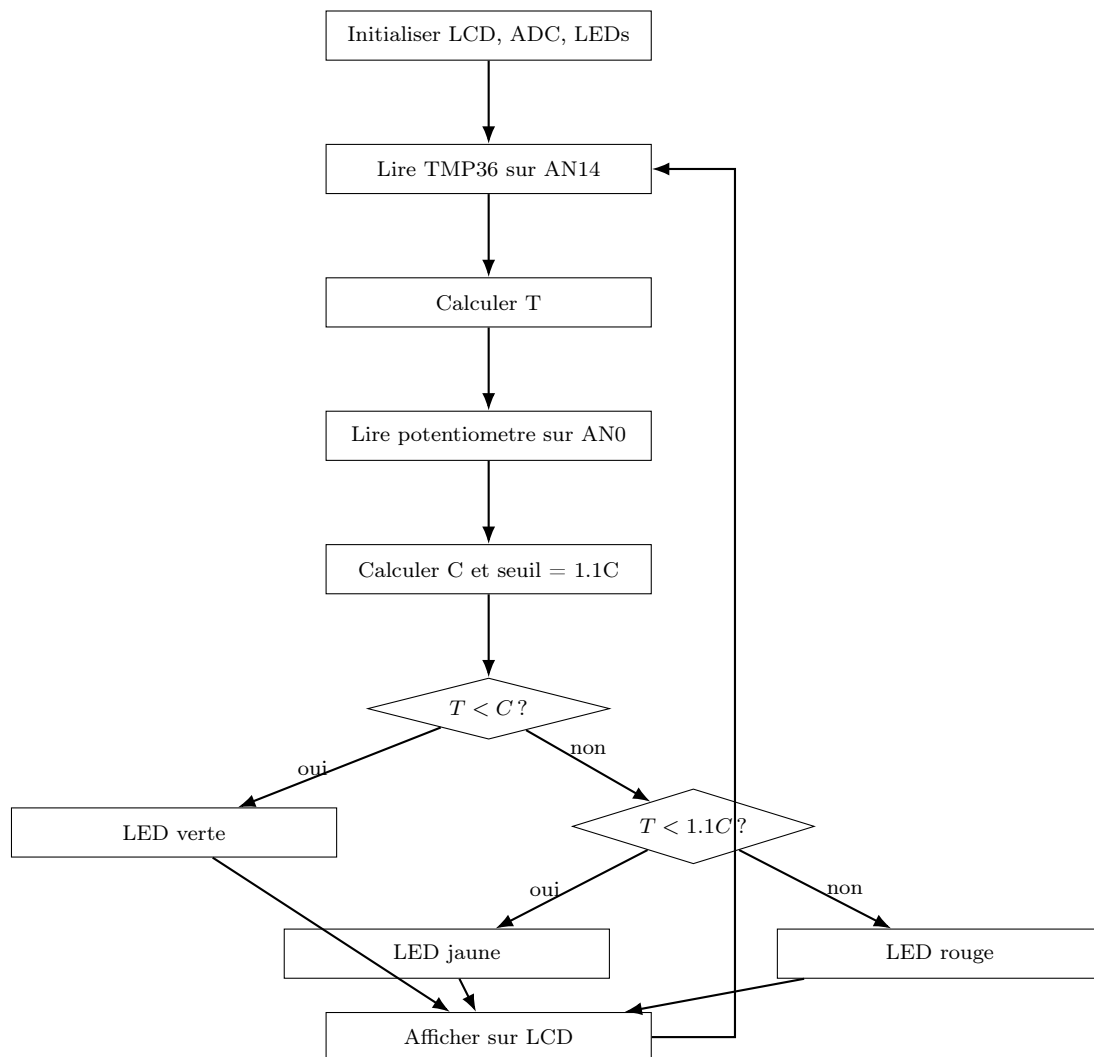


FIGURE 7 – Organigramme noir et blanc du micro-projet sans PWM.

6.5 Code C verifie : micro-projet simple

Le code suivant reprend le mapping définitif et corrige les points observés pendant les tests : utilisation de RC2/AN14 pour le TMP36, RA0/AN0 pour la consigne, et extinction systématique des LEDs avant chaque décision.

Listing 5 – Micro-projet simple : LCD, consigne et LEDs sans PWM

```

1 #include "mcc.h"
2 #include "lcd.h"
3 #include <stdio.h>
4 #include <stdint.h>
5
6 #define LED_JAUNE    LATBbits.LATB3
7 #define LED_VERTE    LATBbits.LATB2
8 #define LED_ROUGE    LATBbits.LATB1
9

```

```

10 #define TMP36_CHANNEL 14 // RC2 = AN14
11 #define POT_CHANNEL 0 // RAO = AN0
12
13 uint16_t Lire_ADC_Moyenne(uint8_t canal)
14 {
15     uint32_t somme = 0;
16     ADCON0bits.CHS = canal;
17
18     for(uint8_t i = 0; i < 8; i++)
19     {
20         __delay_us(50);
21         ADCON0bits.GO_nDONE = 1;
22         while(ADCON0bits.GO_nDONE);
23         somme += ((uint16_t)ADRESH << 8) | ADRESL;
24     }
25     return (uint16_t)(somme / 8);
26 }
27
28 void main(void)
29 {
30     uint16_t adcTemp, adcPot;
31     float voltage, temperature, consigne, seuil_rouge;
32     char buffer[17];
33
34     SYSTEM_Initialize();
35     LCD_Initialize();
36     LCD_Clear();
37
38     ADCON1bits.ADFM = 1;
39
40     TRISCbits.TRISC2 = 1;
41     ANSELbits.ANSC2 = 1;
42
43     TRISAbits.TRISA0 = 1;
44     ANSELABits.ANSA0 = 1;
45
46     TRISBbits.TRISB1 = 0;
47     TRISBbits.TRISB2 = 0;
48     TRISBbits.TRISB3 = 0;
49
50     ANSELBbits.ANSB1 = 0;
51     ANSELBbits.ANSB2 = 0;
52     ANSELBbits.ANSB3 = 0;
53
54     LCD_GoTo(0,0);
55     LCD_WriteString((const uint8_t*)" Micro Projet ");
56     __delay_ms(1000);
57     LCD_Clear();
58
59     while(1)
60     {
61         adcTemp = Lire_ADC_Moyenne(TMP36_CHANNEL);
62         voltage = ((float)adcTemp * 5.0) / 1023.0;
63         temperature = (voltage - 0.5) * 100.0;
64
65         adcPot = Lire_ADC_Moyenne(POT_CHANNEL);
66         consigne = 15.0 + (((float)adcPot * 30.0) / 1023.0);
67         seuil_rouge = consigne * 1.10;
68
69         LED_VERTE = 0;
70         LED_JAUNE = 0;
71         LED_ROUGE = 0;
72
73         if(temperature < consigne)
74         {
75             LED_VERTE = 1;
76         }
77         else if(temperature < seuil_rouge)
78         {

```

```

79         LED_JAUNE = 1;
80     }
81     else
82     {
83         LED_ROUGE = 1;
84     }
85
86     LCD_Clear();
87     LCD_GoTo(0,0);
88     sprintf(buffer,"T=%.1fC C=%.1fC",temperature,consigne);
89     LCD_WriteString((const uint8_t*)buffer);
90
91     LCD_GoTo(1,0);
92     if(temperature < consigne)
93         LCD_WriteString((const uint8_t*)"LED VERTE");
94     else if(temperature < seuil_rouge)
95         LCD_WriteString((const uint8_t*)"LED JAUNE");
96     else
97         LCD_WriteString((const uint8_t*)"LED ROUGE");
98
99     __delay_ms(500);
100 }
101 }

```

7 Extension PWM

7.1 Correction de la logique PWM

La logique PWM doit uniquement etre activee lorsque la temperature depasse de plus de 10% la consigne. L'ordre coherent est donc :

1. si $T < C$: LED verte allumee;
2. sinon si $T < 1.1C$: LED jaune allumee;
3. sinon : LED rouge commandee en PWM.

Il ne faut donc pas appeler la PWM dans la branche $T < 1.1C$. Cette branche correspond a un depassement simple de consigne, pas a l'alarme critique. La branche PWM est uniquement la derniere branche.

7.2 Principe de la PWM

La PWM fait varier le rapport cyclique d'un signal periodique. Pour une LED, cela modifie l'intensite lumineuse apparente. Dans le projet, plus le depassement au-dessus du seuil critique est grand, plus le rapport cyclique est eleve.

$$D = \frac{T - 1.1C}{5} \times 1023$$

La valeur est limitee a 1023 pour respecter la resolution du module PWM.

7.3 Routage PPS vers RB1

Le PIC16F1719 utilise le PPS pour associer une sortie peripherique a une broche physique. Ici, la sortie PWM2 est associee a RB1 par :

Listing 6 – Routage PPS de PWM2 vers RB1

```
1 RB1PPS = 0x0D;    // PWM2 vers RB1
```

7.4 Code C final avec logique PWM corrigée

Listing 7 – Micro-projet final avec LED rouge PWM

```
1 #include "mcc.h"
2 #include "lcd.h"
3 #include "pwm2.h"
4 #include "tmr2.h"
5 #include <stdio.h>
6 #include <stdint.h>
7
8 #define LED_JAUNE    LATBbits.LATB3
9 #define LED_VERTE    LATBbits.LATB2
10 #define LED_ROUGE    LATBbits.LATB1
11
12 #define TMP36_CHANNEL    14    // RC2 = AN14
13 #define POT_CHANNEL      0    // RAO = AN0
14
15 uint16_t Lire_ADC_Moyenne(uint8_t canal)
16 {
17     uint32_t somme = 0;
18     ADCON0bits.CHS = canal;
19
20     for(uint8_t i = 0; i < 8; i++)
21     {
22         __delay_us(50);
23         ADCON0bits.GO_nDONE = 1;
24         while(ADCON0bits.GO_nDONE);
25         somme += ((uint16_t)ADRESH << 8) | ADRESL;
26     }
27     return (uint16_t)(somme / 8);
28 }
29
30 void PWM_Rouge_Enable(void)
31 {
32     TMR2_StartTimer();
33
34     PPSLOCK = 0x55;
35     PPSLOCK = 0xAA;
36     PPSLOCKbits.PPSLOCKED = 0;
37
38     RB1PPS = 0x0D;    // PWM2 vers RB1
39
40     PPSLOCK = 0x55;
41     PPSLOCK = 0xAA;
42     PPSLOCKbits.PPSLOCKED = 1;
43 }
44
45 void PWM_Rouge_Disable(void)
46 {
47     PWM2_LoadDutyValue(0);
48
49     PPSLOCK = 0x55;
50     PPSLOCK = 0xAA;
51     PPSLOCKbits.PPSLOCKED = 0;
52
53     RB1PPS = 0x00;    // RB1 redevient GPIO
54
55     PPSLOCK = 0x55;
56     PPSLOCK = 0xAA;
57     PPSLOCKbits.PPSLOCKED = 1;
58 }
```

```

59     LATBbits.LATB1 = 0;
60 }
61
62 void main(void)
63 {
64     uint16_t adcTemp, adcPot;
65     float voltage, temperature, consigne, seuil_rouge;
66     char buffer[17];
67
68     SYSTEM_Initialize();
69     LCD_Initialize();
70     LCD_Clear();
71
72     ADCON1bits.ADFM = 1;
73
74     TRISCbits.TRISC2 = 1;
75     ANSELbits.ANSC2 = 1;
76
77     TRISAbits.TRISA0 = 1;
78     ANSELbits.ANSA0 = 1;
79
80     TRISBbits.TRISB1 = 0;
81     TRISBbits.TRISB2 = 0;
82     TRISBbits.TRISB3 = 0;
83
84     ANSELBbits.ANSB1 = 0;
85     ANSELBbits.ANSB2 = 0;
86     ANSELBbits.ANSB3 = 0;
87
88     LED_JAUNE = 0;
89     LED_VERTE = 0;
90     LED_ROUGE = 0;
91     PWM2_LoadDutyValue(0);
92
93     LCD_GoTo(0,0);
94     LCD_WriteString((const uint8_t*)"  Micro Projet  ");
95     __delay_ms(1000);
96     LCD_Clear();
97
98     while(1)
99     {
100         adcTemp = Lire_ADC_Moyenne(TMP36_CHANNEL);
101         voltage = ((float)adcTemp * 5.0) / 1023.0;
102         temperature = (voltage - 0.5) * 100.0;
103
104         adcPot = Lire_ADC_Moyenne(POT_CHANNEL);
105         consigne = 15.0 + (((float)adcPot * 30.0) / 1023.0);
106         seuil_rouge = consigne * 1.10;
107
108         LED_VERTE = 0;
109         LED_JAUNE = 0;
110         LED_ROUGE = 0;
111
112         if(temperature < consigne)
113         {
114             PWM_Rouge_Disable();
115             LED_VERTE = 1;
116         }
117         else if(temperature < seuil_rouge)
118         {
119             PWM_Rouge_Disable();
120             LED_JAUNE = 1;
121         }
122         else
123         {
124             uint16_t dutyCycle;
125             float depassement;
126
127             LED_VERTE = 0;

```

```

128     LED_JAUNE = 0;
129
130     PWM_Rouge_Enable();
131
132     depassement = temperature - seuil_rouge;
133     if(depassement > 5.0)
134         depassement = 5.0;
135
136     dutyCycle = (uint16_t)((depassement / 5.0) * 1023.0);
137     PWM2_LoadDutyValue(dutyCycle);
138 }
139
140 LCD_Clear();
141 LCD_GoTo(0,0);
142 sprintf(buffer, "T=%.1fC C=%.1fC", temperature, consigne);
143 LCD_WriteString((const uint8_t*)buffer);
144
145 LCD_GoTo(1,0);
146 if(temperature < consigne)
147     LCD_WriteString((const uint8_t*)"LED VERTE");
148 else if(temperature < seuil_rouge)
149     LCD_WriteString((const uint8_t*)"LED JAUNE");
150 else
151     LCD_WriteString((const uint8_t*)"ROUGE PWM");
152
153 __delay_ms(500);
154 }
155 }

```

8 Configurations utilisees

8.1 Fichiers MCC et bibliotheques

Les fichiers utilises sont :

- `mcc.h` : initialisation generale du systeme ;
- `lcd.h` et `lcd.c` : fonctions de commande du LCD ;
- `pwm2.h` et `pwm2.c` : pilotage du module PWM2 ;
- `tmr2.h` et `tmr2.c` : base de temps du module PWM ;
- `adc.h` et `adc.c` : configuration et lecture de l'ADC.

8.2 Registres importants

Registre	Role dans le projet
<code>TRISCbits.TRISC2</code>	Configure RC2 comme entree pour le TMP36.
<code>ANSELbits.ANSC2</code>	Active la fonction analogique sur RC2/AN14.
<code>TRISA0bits.TRISA0</code>	Configure RA0 comme entree pour le potentiometre.
<code>ANSELABits.ANSA0</code>	Active la fonction analogique sur RA0/AN0.
<code>TRISBbits.TRISB10</code>	Configure les broches des LEDs en sortie.
<code>ANSELBbits.ANSB10</code>	Active l'analogique sur les broches LEDs.
<code>ADCON0bits.CHS</code>	Selectionne le canal ADC a convertir.

ADCON1bits.ADFM	Justifie a droite le resultat ADC 10 bits.
RB1PPS	Route PWM2 vers RB1.
PWM2_LoadDutyValue	Fixe le rapport cyclique de la PWM.

9 Schemas et cablage

Le montage final utilise le TMP36 sur une entree analogique separee du potentiometre de la carte. Les tests ont montre que RC2/AN14 donnait une mesure stable et independante de la consigne. Les LEDs sont reliees a RB1, RB2 et RB3 via des resistances de limitation.

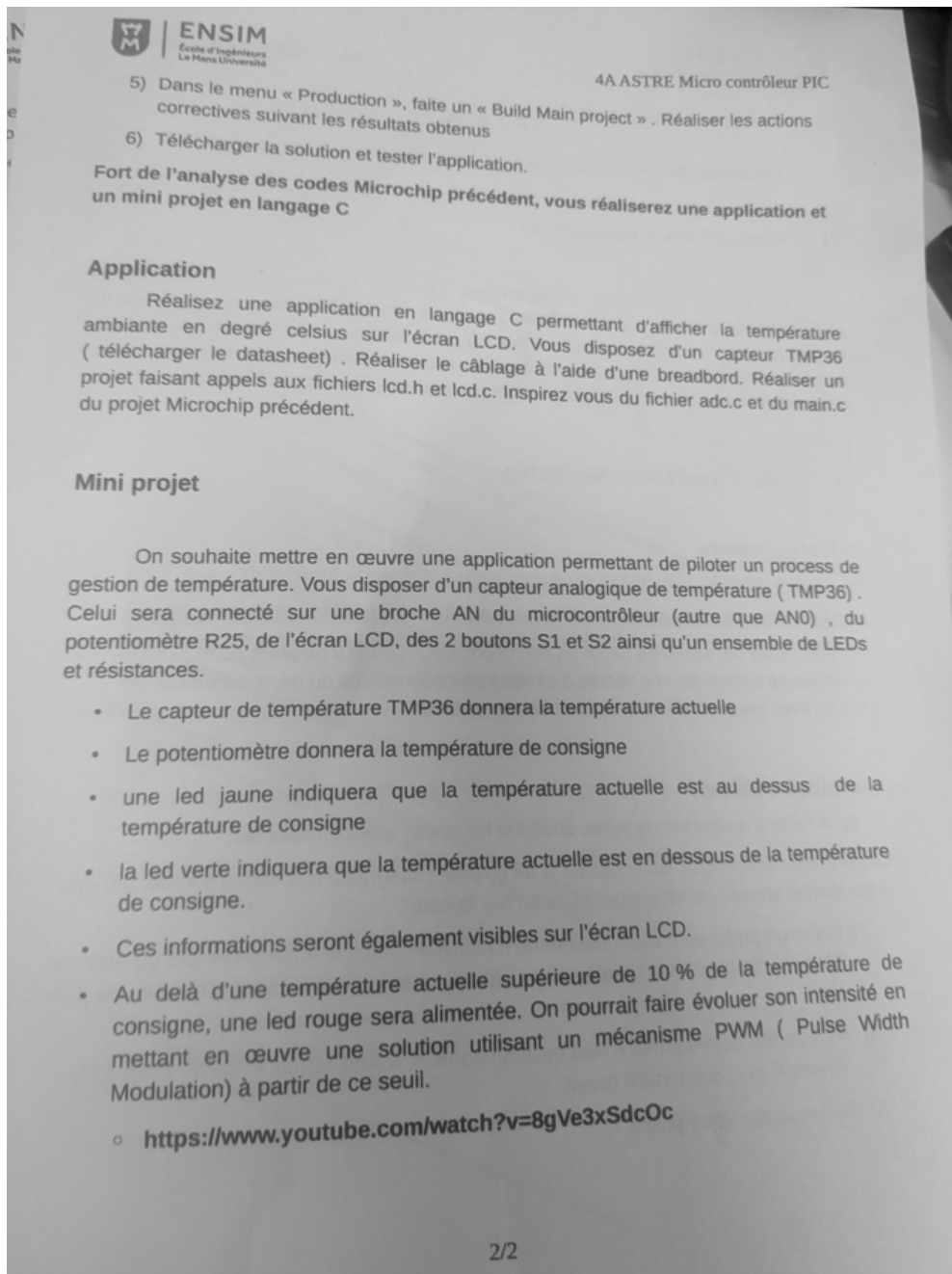


FIGURE 8 – Sujet de l'application et du micro-projet.

Initialisation

```

├─ LCD
├─ ADC
├─ PWM
├─ LEDs

```

Boucle infinie

```

├─ Lire AN1 (TMP36)
├─ Convertir en °C

├─ Lire AN0 (potentiomètre)
├─ Convertir en température de consigne
█
├─ Afficher sur LCD

├─ Si  $T < T_c$ 
│   LED verte ON
│   LED jaune OFF

```

```

├─ PWM = 0

├─ Si  $T > T_c$ 
│   LED jaune ON
│   LED verte OFF

├─ Si  $T > T_c + 10\%$ 
│   LED rouge PWM

└─ Recommencer

```

FIGURE 9 – Prototype d'algorithme fourni pendant le TP.

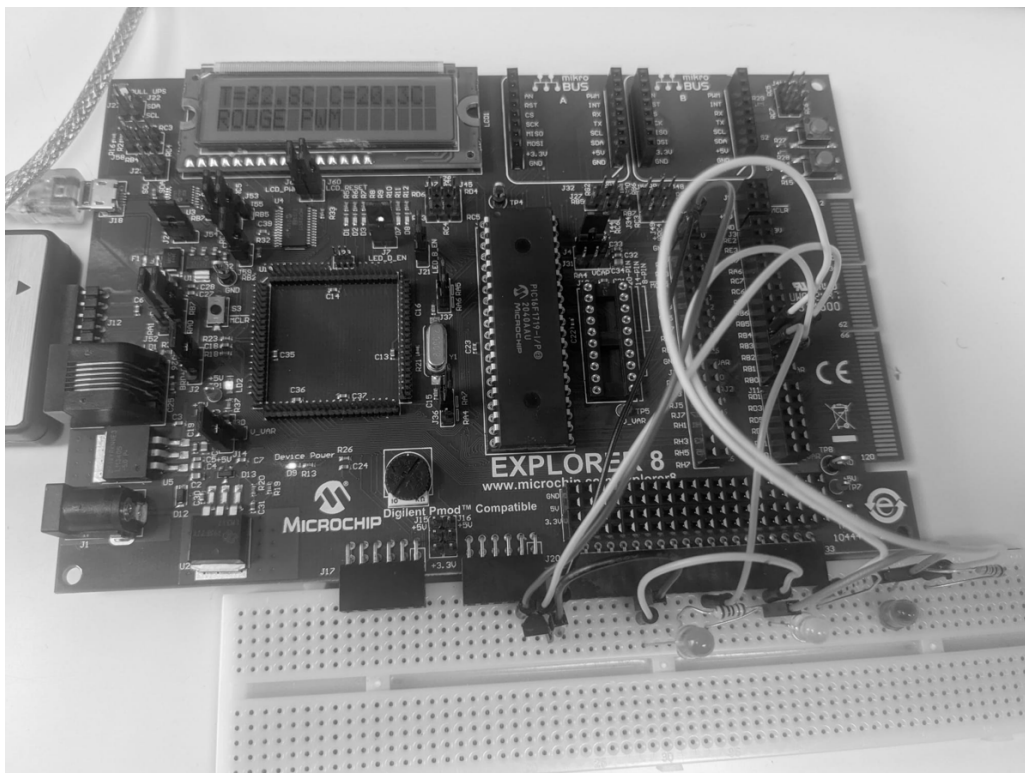


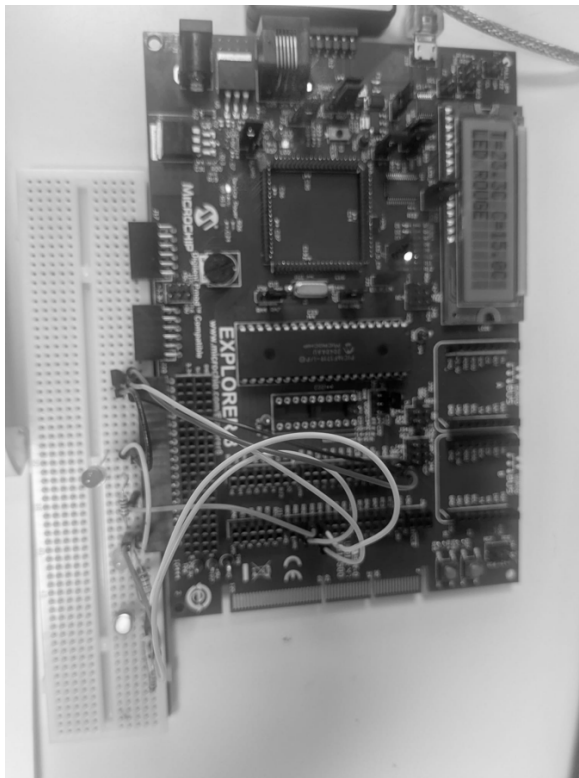
FIGURE 10 – Vue d'ensemble du montage final sur Explorer 8 et breadboard.

10 Analyse des resultats experimentaux

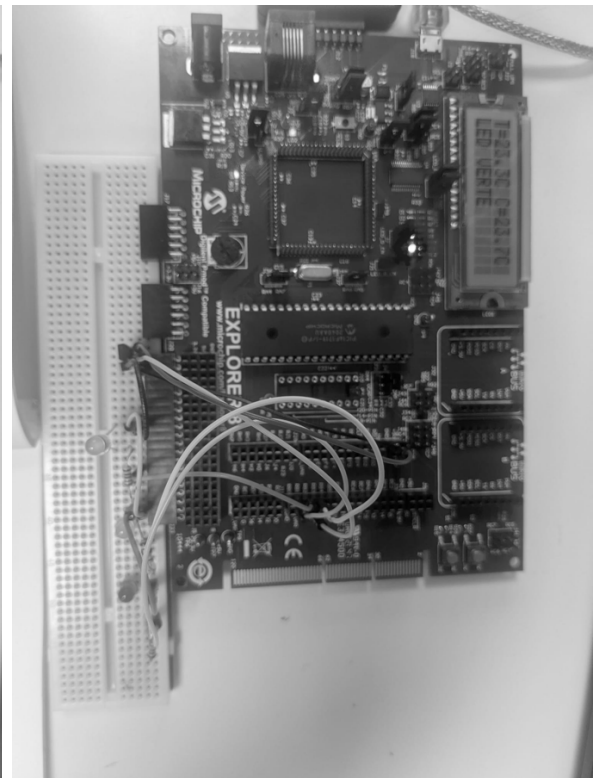
10.1 Affichage LCD

Les captures montrent les messages LCD typiques :

- LED VERTE lorsque la temperature est inferieure a la consigne ;
- LED JAUNE lorsque la temperature depasse la consigne mais reste sous le seuil rouge ;
- ROUGE PWM lorsque la temperature depasse le seuil critique.

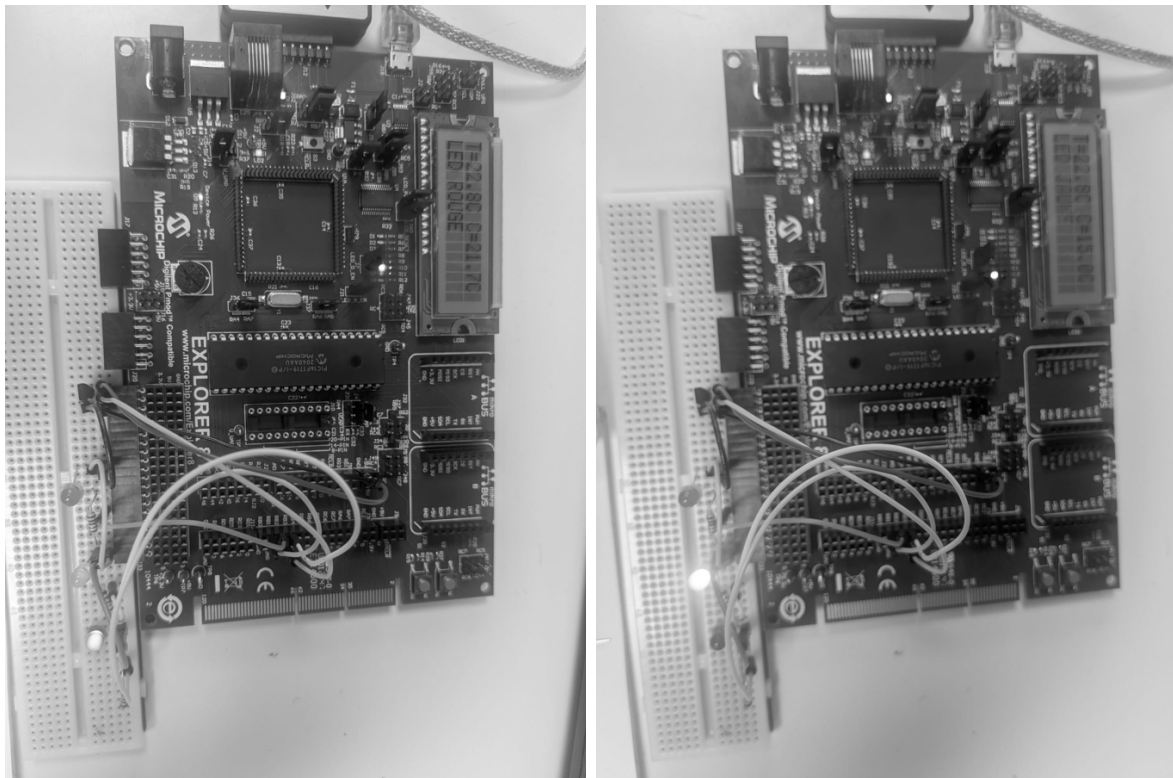


(a) Etat rouge observe.



(b) Etat vert observe.

FIGURE 11 – Captures d'état du système sur la carte.



(a) Affichage LCD avec alarme rouge.

(b) Affichage LCD avec LED jaune.

FIGURE 12 – Resultats LCD et LEDs.

10.2 Cohérence avec la position du potentiometre

La consigne affichee sur le LCD varie avec la position du potentiometre. Plus le potentiometre est tourne vers la tension maximale, plus la valeur ADC lue sur AN0 augmente, et plus la consigne calculee se rapproche de 45 °C. Inversement, lorsque le potentiometre est proche du minimum, la consigne se rapproche de 15 °C. Les etats observes sont coherents :

- si la consigne est elevee, la temperature ambiante est souvent inferieure : LED verte ;
- si la consigne est basse et que la temperature est superieure, le systeme passe en jaune ou en rouge ;
- si le seuil de 10% est depasse, la LED rouge est commandee par PWM.

11 Points de verification et corrections realisees

11.1 Probleme de variation de temperature avec la consigne

Pendant les essais, la temperature affichee variait lorsque le potentiometre etait tourne. Cela indiquait que la lecture TMP36 et la lecture potentiometre n'etaient pas correctement separees. Les corrections retenues sont :

- utiliser RC2/AN14 pour le TMP36 ;

- conserver RA0/AN0 pour le potentiometre ;
- faire un moyennage de 8 conversions ;
- laisser un delai d'acquisition de 50 μ s avant chaque conversion.

11.2 Correction de la logique des LEDs

Le texte initial melangeait parfois la LED jaune et la LED rouge. La logique finale conserve la coherence suivante :

$$\begin{cases} T < C & \Rightarrow \text{LED verte} \\ C \leq T < 1.1C & \Rightarrow \text{LED jaune} \\ T \geq 1.1C & \Rightarrow \text{LED rouge PWM} \end{cases}$$

12 Conclusion

Le projet a permis de mettre en oeuvre plusieurs notions importantes de programmation sur PIC16F1719 : configuration des broches, lecture ADC, conversion physique d'une tension en temperature, affichage LCD, pilotage de LEDs et generation PWM. Le principal point critique etait le choix de l'entree analogique du TMP36. Les tests ont montre que l'utilisation de RC2/AN14 permettait d'obtenir une mesure plus stable et independante du potentiometre. La logique finale respecte les exigences du sujet et relie correctement l'application de base au micro-projet avec PWM.

References documentaires

- Microchip, *Explorer 8 Development Board User's Guide*, DS40001812B.
- Microchip, *PIC16(L)F1717/8/9 Data Sheet*, DS40001740B.
- Analog Devices, *TMP35/TMP36/TMP37 Low Voltage Temperature Sensors*, Rev. C.